Crop classifier pipeline (imbalanced, tiny crops)

Script: `insect_classifier.py`. Context: `../../CONTEXT.md`, dataset guide: `../step2-3-dataset-and-roboflow.md`.

Problem: ~500-600 crops, 10×10 to 100×100 px, with few `stink_bug` / `chinch_bug` / `wireworm` crops and many `other_insect` crops.

Where it fits

This is the **second stage** of a two-stage system: a detector finds every insect (box), then this model classifies each crop. Counts are the per-class totals of the classifier's answers. It is the alternative to a single 4-class Roboflow detector. Train both and compare them on the same held-out photos; that comparison is a good judging slide. If the 4-class detector already gets the targets right, you don't need this.

Run it (laptop, ~10-20 min on CPU, faster with a GPU or Apple MPS)

```
python -m venv venv && source venv/bin/activate
pip install torch torchvision pillow onnx onnxruntime

# 1. Roboflow: Versions -> Export -> format "YOLOv8" ->
download zip, unzip to roboflow_export/
#    Export with NO augmentation (the script augments)
and NO resize if possible.
python insect_classifier.py crops --export
roboflow_export/ --out crops/
#    ->
```

```
crops/{train,valid,test}/{stink_bug,chinch_bug,wireworm,
other_insect}/*.png
#    (keeps Roboflow's per-photo split, so crops of one
photo never leak into test)

# 2. Train (pretrained ImageNet weights are downloaded
the first time)
python insect_classifier.py train --data crops/ --out
runs/r18 --arch resnet18

# 3. Try a variant or two and compare macro-F1 / target
recall on TEST
python insect_classifier.py train --data crops/ --out
runs/r18_focal --balance focal
python insect_classifier.py train --data crops/ --out
runs/mnv3 --arch mobilenet_v3_large

# 4. Export the winner for the Pi
python insect_classifier.py export --ckpt
runs/r18/best.pt --out runs/r18/classifier.onnx
```

Already have crops in plain `crops/<class>/` folders? Point `--data` at that. The script then splits 70/15/15 **by source photo**, using the part of the filename before `__`.

What the pipeline does, and why

Problem	What the script does
Tiny crops (10 px)	Crops are cut with 15% padding (keeps legs and some context), then **letterboxed to 96×96**: the aspect ratio is kept (wireworms are long and thin) and small crops are upscaled.

Problem	What the script does
Size is informative	The crop's original size (log of the longest side) is fed to the model next to the image features. The camera is at a fixed distance, so a 12 px dark blob being a chinch bug is a real cue. Turn it off with `--no-size` to compare.
Severe imbalance	Default `--balance sampler`: a **weighted sampler** shows rare classes more often (weights = inverse frequency^0.75). Alternatives: `weights` (class-weighted cross-entropy) or `focal` (focal loss). Label smoothing 0.05.
Few images	**ImageNet-pretrained backbone** (ResNet18 default; MobileNetV3 or EfficientNet-B0 for a lighter Pi model). Heavy but realistic augmentation: flips, 90° rotations, ±15° rotation, brightness/contrast/colour jitter (lighting robustness), blur, random down-scaling (simulates the smallest insects).
Picking the model	Early stopping on **validation macro-F1**, not accuracy. Accuracy looks great when the model just says "other" every time.
Rare classes under-called	After training, the script tunes a single **bias on the `other_insect` score** on validation and applies it to test and export. It only moves if macro-F1 improves by at least 0.01.

Problem	What the script does
Honest result	Prints per-class precision/recall/F1 and a **confusion matrix** on the held-out test split. Saves everything to `metrics.json`.

Reading the output

- Most important: **recall for the three targets** on TEST, and the confusion rows for chinch_bug vs other_insect (look-alikes: big-eyed bug, ataenius, springtail).
- If one target's recall is low: add crops of that class (shoot more photos with it), try `--balance focal` or `--power 1.0`, or `--size 128`.
- If decoys get called targets (low precision): add more photos with those decoys.
- Fewer than ~20 test crops per target makes the numbers noisy. Say that in the presentation rather than over-claiming.

On the Pi

`classifier.onnx` plus `classifier.json` (classes, input size, `other_bias`). For each detected box: crop with the same 15% padding, letterbox to the input size, normalise with the ImageNet mean/std, pass `size = (ln(max(w,h)) − ln 10) / (ln 100 − ln 10)`, add `other_bias` to the `other_insect` logit, then take the argmax. Run with `onnxruntime` (`pip install onnxruntime` works on Pi 5 64-bit).

Tested

Ran end-to-end on a synthetic, imbalanced Roboflow-style export (Fri Oct 23): crop extraction, training with sampler and focal loss, flat-folder split, bias tuning and ONNX export (onnxruntime loads it). This was not tested on real BugBytes photos yet.